

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_f9aa1d1c-c83\agent-aeaa3e878f1d80b90.jsonl`  
- SHA-256: `7b50146ab7c1fb3712a1049f66b27ad7ee74c6ea3662a6609c214271b5caed62`  
- Source modified: `2026-06-09T11:39:52+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_f9aa1d1c-c83`  
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-09T11:25:35.356Z)

DIMENSION = CONVERSATIONAL VIDEO-QA ARCHITECTURE ("upload a video, then ask contextual questions continuously": longest rally, all service faults, rally>20 shots, 'make a clip of X'). Use WebSearch+WebFetch for patterns (video agents, structured-event indexes, retrieval over per-video metadata). KEY design question: should the heavy model do everything, or should we EXTRACT structured events (rallies/shots/faults with timestamps) into a PER-VIDEO event store, then answer questions with a lighter query/conversational layer over that store (+ ffmpeg to cut clips)? Assess: feasible to DESIGN for now (build the bookkeeping/state layer into the framework) vs PUNT the full conversational model. What state-management does it imply?

OWNER CONTEXT:

OWNER'S REFRAMED VISION (from PR #61 review comments ? supersedes earlier framing where they differ):

- IMMEDIATE GOAL: a TRAINING FRAMEWORK + HARNESS to produce a VERY HIGH precision/recall model that extracts

- HIGHLIGHTS and RALLIES from a clip. Quality first. On-device is LATER (a model-compression goal for power users to

- save upload cost) ? NOT the immediate goal; don't over-index on it now.

- MULTISPORT FROM THE START: all NET-SEPARATED RACQUET SPORTS (badminton, tennis, table tennis, pickleball, padel)

- first, eventually ALL sports. A SINGLE, sports-AGNOSTIC model (the collector + indexer are already sport-generic by

- design; the repo will be renamed off "badminton"). More labels/compute/storage is an accepted "good problem."

- CONVERSATIONAL VIDEO-QA is a north-star feature: upload a video -> the model processes it -> the user asks CONTEXTUAL

- QUESTIONS continuously, e.g. "show me the longest rally", "make a video of all the service faults in this clip",

- "show me a clip where the rally lasted over 20 shots". This implies per-video

- STATE/bookkeeping the model itself

- won't hold ? so the FRAMEWORK must be designed for that bookkeeping from the start. Owner asks: feasible to embark on

- now, or punt and revisit after quality is solid?

- HARD COMMERCIAL-LICENSE GUARDRAIL (owner: "never corrupted"): EVERY dependency ? code, data, AND model weights ? must

- be usable in PRIVATE PROPRIETARY COMMERCIAL software. MIT / Apache-2.0 / BSD are the easy fits. Reject viral/NC/

- custom-restrictive (e.g. Gemma-3 viral terms; weights trained on paid-LLM outputs; >X-MAU caps if truly binding).

- NEVER train on paid-LLM (Gemini/OpenAI/Anthropic) outputs. Exclude minors; separate training opt-in. Hopefully a

- CONVERSATIONAL single model.

- CURRENT STATE (de-risking evidence): on a 6-match human-labeled corpus, a learned per-frame prototype gets held-out

- AUC 0.83-0.92 and a cross-video LOVO mean 0.583 vs the heuristic's 0.480 ? and the learned

model DECOUPLES from the

multi-court "background games" confound the heuristic can't escape. WASB = the shuttle detector (MIT code, but the

distributed checkpoint is academic-data-trained ? provenance unresolved = a commercial-cleanliness blocker to fix).

- CONSTRAINTS: solo founder, Bangalore; local dev box = ONE RTX 2070 Super (8GB) + WSL2; paid Gemini stays the fallback;

cloud GPU rented per-job (Modal/RunPod per the earlier study).

REQUIREMENTS BRIEF:

```
{"requirements":["GOAL (reframed): The immediate deliverable is a TRAINING FRAMEWORK + HARNESS that produces a VERY high precision/recall model for extracting HIGHLIGHTS and RALLIES from a clip. Quality is the first-order objective; everything else is secondary to model quality.", "ON-DEVICE IS DEFERRED (reframe vs earlier 'on-device = immediate moat'): On-device / model-compression is a LATER goal, motivated as an upload-cost saver for power users. It is NOT the immediate target and the strategy must NOT over-index on it now. (Supersedes OWNED_MODEL_IMPLEMENTATION_PLAN.md's 'on-device = the privacy moat / immediate' framing.)", "MULTISPORT FROM DAY ONE: Build for ALL net-separated racquet sports first (badminton, tennis, table tennis, pickleball, padel), with a path to eventually ALL sports. No single-sport (badminton-only) scoping.", "SINGLE SPORT-AGNOSTIC MODEL: Target ONE sports-agnostic model rather than per-sport models. Collector + indexer are already sport-generic by design; the repo will be RENAMED off 'badminton'. Generic data schema stays; coach->athlete remains an optional overlay, not baked in.", "SCALE COST ACCEPTED: More labels / compute / storage from multisport breadth is an explicitly accepted 'good problem' ? do not constrain the design to minimize label/compute/storage volume at the expense of breadth or quality.", "CONVERSATIONAL VIDEO-QA IS THE NORTH-STAR FEATURE: upload a video -> model processes it -> user asks CONTEXTUAL questions CONTINUOUSLY (e.g. 'show me the longest rally', 'make a video of all the service faults', 'show me a clip where the rally lasted over 20 shots'). Hopefully served by a single conversational model.", "PER-VIDEO STATE/BOOKKEEPING IS A FRAMEWORK REQUIREMENT: The conversational-QA loop implies durable per-video state (rally inventory, shot counts, event/fault tags, timestamps, derived clips) that the MODEL ITSELF will not hold. The FRAMEWORK must be DESIGNED FOR THIS BOOKKEEPING FROM THE START, even if the QA feature ships later.", "OPEN DECISION the owner is asking for a recommendation on: is conversational-QA feasible to EMBARK ON NOW, or should it be PUNTED and revisited AFTER rally/highlight quality is solid? (The bookkeeping-from-the-start requirement holds either way.)", "HARD COMMERCIAL-LICENSE GUARDRAIL ('never corrupted'): EVERY dependency ? code, training data, AND model weights ? must be usable in PRIVATE, PROPRIETARY, COMMERCIAL software. MIT / Apache-2.0 / BSD are the accepted easy fits.", "REJECT viral / non-commercial / custom-restrictive licenses: explicitly reject Gemma-3 viral terms, weights trained on paid-LLM outputs, and >X-MAU caps IF truly binding for commercial use.", "NEVER TRAIN ON PAID-LLM OUTPUTS: Gemini / OpenAI / Anthropic outputs must never be a training-data source (terms/copyright). Paid Gemini remains inference/serving/FALLBACK only. (Also: the existing Gemini-silver training labels in training.label_candidates must be purged before they bake in ? M0.3.)", "DATA-ELIGIBILITY GUARDRAILS: Exclude minors from the training pool; per-user training data requires a SEPARATE explicit opt-in. Split data by match.", "QUALITY EVIDENCE / STARTING POINT (de-risking baseline to beat): On a 6-match human-labeled corpus, the learned per-frame prototype achieves held-out AUC 0.83-0.92 and cross-video LOVO mean 0.583 vs the heuristic's 0.480. The learned model is the chosen path BECAUSE it DECOUPLES from the multi-court 'background games' confound that the heuristic cannot escape (3 heuristic levers ? spatial gate, audio-foreground, court-region separability ? were each tested and failed this session; the problem is temporal, not spatial).", "WASB PROVENANCE IS AN OPEN COMMERCIAL-CLEANLINESS BLOCKER: WASB is the shuttle/ball detector (MIT code), but the distributed checkpoint is academic-data-trained with unresolved provenance (the C3 item) ? this is a commercial-cleanliness blocker that must be resolved (document as Gen-0 prerequisite + plan own-weights, or scope an alternative).", "RESOURCE/OPERATING CONSTRAINTS: Solo founder, Bangalore. Local dev box = ONE RTX 2070 Super (8GB VRAM) + WSL2 (no full-pipeline run in the dev container ? no GPU/torch/cv2/numpy there). Cloud GPU rented per-job (Modal / RunPod per the earlier study). Paid Gemini stays the fallback.", "LANGUAGE/SCOPE NOTE: The reframed vision adds no new natural-language/multilingual requirement beyond English conversational-QA phrasing; 'language' as a strategic axis is limited to the conversational-QA query interface, not localization.", "tensions":["QUALITY-FIRST vs ON-DEVICE-LATER vs HARDWARE: The very-high-quality goal pushes toward larger VLM bases and cloud training, but the only local box is an 8GB RTX 2070 Super and on-device is explicitly deferred ? so the immediate model is NOT constrained to fit 8GB, yet the eventual compression goal must remain reachable. The strategy must pick a base that is both high-quality-trainable AND later-compressible without re-architecting.", "MULTISPORT-FROM-START vs CURRENT EVIDENCE DEPTH: The de-
```

risking corpus is badminton-only (WASB shuttle features, multi-court badminton confound). A single sport-agnostic model is required from day one, but the only validated signal/features and the only labeled corpus are badminton ? the feature stack (WASB trajectory) may not generalize to tennis/table-tennis/padel, and per-sport corpora don't exist yet.", "SINGLE SPORT-AGNOSTIC MODEL vs COMMERCIAL-CLEAN DETECTORS: A sport-agnostic model likely needs per-sport object/ball detectors or a unified detector; WASB (badminton) already has unresolved provenance (C3), and clean MIT/Apache/BSD detectors for tennis/TT/pickleball/padel are not yet identified ? each new sport reopens the license-cleanliness guardrail.", "CONVERSATIONAL-QA NORTH-STAR vs 'NEVER TRAIN ON PAID-LLM': A single conversational model that answers contextual video questions typically wants an LLM/VLM teacher, but the strongest available teachers (Gemini/OpenAI/Anthropic) are forbidden as a training source ? so the conversational capability must come from an Apache-2.0/MIT base + own data, narrowing the candidate set and raising the quality bar to clear without paid-LLM distillation.", "BOOKKEEPING-FROM-THE-START vs SEQUENCING DISCIPLINE: The owner wants the framework designed for per-video QA state NOW, but the committed next slice is the async job model (single-tenant) and heavy model work is gated on scaffolding + sign-off ? designing durable QA-state bookkeeping prematurely risks building against an architecture (and a model) that doesn't exist yet (the 'embark now vs punt' question the owner is explicitly asking).", "ACCEPTED SCALE COST vs SOLO-FOUNDER / SINGLE-GPU BUDGET: 'More labels/compute/storage is a good problem' collides with a solo founder on one 8GB GPU renting cloud per-job ? multisport label volume and storage are accepted in principle but still bounded by one person's labeling throughput and per-job cloud spend.", "SINGLE CONVERSATIONAL MODEL' (hoped) vs HARNESS-FIRST QUALITY GOAL: The owner hopes for one model that does detection + segmentation + conversational QA, but the harness/quality goal is concretely about a rally/highlight extractor; folding conversational QA into the same model vs keeping extraction (model) and QA (bookkeeping + retrieval/LLM layer) separate is an unresolved architectural fork.", "QUALITY-FIRST ('very high precision/recall') vs HONEST EVIDENCE CEILING: Current best cross-video LOVO is ~0.583 (directional, n<=6, linear, threshold-transfer unsolved) ? a large gap from 'very high precision/recall', so the quality target may require substantially more data/features/base-model than the current de-risking artifacts demonstrate."]]

2. assistant (2026-06-09T11:25:39.770Z)

I'll research conversational video-QA architecture patterns to answer the owner's key question: should the heavy model do everything, or should we extract structured events into a per-video event store and answer questions with a lighter query layer? Let me start by examining the existing codebase to ground my recommendation, then research external patterns.

3. user (2026-06-09T11:25:41.096Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-255 of 330 total (27423 tokens, cap 25000). Call Read with offset=256 limit=255 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
4      hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
5      segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
6
7      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
8      doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
9      (this revision follows #40).
10
11     **LEGEND**
12     - `@path` = file (always repo-relative)
13     - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
14     - `->` = dataflow / call / "produces"
15     - `$` = data contract (canonical shape)
16     - `?` = gotcha / footgun
17     - `?` = registry / plugin extension point
18     - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
19
20     ---
21
```

```

18     ## 1. PIPELINE
19
20     ### 1A. Runtime: video file -> highlight reel
21     ```
22     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
23                                     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
24     -> video_id = MD5(whole file)          @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
25     -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
26     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
27     [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
28     -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
29     -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
30     -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
31     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
32     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
33     -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
34     -> select segment ids -> [(start,end)] (+ stitching padding)
35     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
36     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
37     ```
38     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
39     ```
40     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
41     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
42     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
43     -> P4 db.add_play_segment + db.link_candidate_to_segment
44     ```
45     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
46     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
47
48     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
49     ```
50     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
51     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
52     -> WasbRunner.run_predict(video, out_dir)
53     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
54     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
55     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
56     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING

```

```

BRAIN]
57     ...
58     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
59
60     ### 1C. Calibration / eval flow (NO pipeline run unless told)
61     ...
62     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
63     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]    stage
? {candidates, final}
64     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
65     -> harness.evaluate -> EvalReport -> report_to_dict
66     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
67     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
68     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
69     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
70     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
71     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
72     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
73     ...
74     Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
75
76     ---
77
78     ## 2. SUBSYSTEMS
79
80     ### 2.0 Orchestration & config
81     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
82     - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
83     - `::process_video` `@POST /api/process` -> validate -> add_video -> segment ->
`finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST
/api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`.
`::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read).
84     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
85     - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
86     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
87     - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
88     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.

```

```

`--stage{candidates,final,both}`. Exit 2=arg/I0. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89 - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
90 - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
91
92 - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93 - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
94 - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config` = {extra:'allow', validate_assignment:True}. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
96 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)` .
97 - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`,
`min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config` = `AppConfig` alias.
98 - `@backend/config/loader.py` ? load/save with defaults-merge.
99 - `::DEFAULT_CONFIG_PATH` = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_built_in_defaults` = `AppConfig().model_dump(...)` .
100 - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101 - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
102 - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
103 - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105 ### 2.1 Segmenters (under pipeline/segmenters) ?
106 - `@backend/pipeline/segmenters/base.py`

```

```

107     - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
108     - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109     - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110     - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
111     - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112     - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::PERSON_CLASS_ID=1` ? (ultraalytics used 0; wrong value -> zero detections).
`::lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::extract_action_windows_from_chunk`, `::make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113     - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114     - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
115
116     ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117     - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool, str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
118     - `@backend/pipeline/detectors/tracknet_runner.py`
119     - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120     - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled),
`::TrajectoryPoint` (frame, visible, x, y).
121     - `*::parse_trajectory_csv` @staticmethod + `*::trajectory_to_action_windows`
@staticmethod = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.

```

```

122     - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+:`WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `

```



```

`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
135     - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
136     - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON "model":"logreg" tag.
137     - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPS ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
138     - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO.
`::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT
fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments`
(smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC
+ LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer
unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage**
(Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/
calibrate_local.
139     - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally+dead-
time visibility "contrast" (?3.6x heuristic works, ?1.9x fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140     - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
141     - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
142     - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144     ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145     - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2x, band-pass null -> NOT adopted
(PR #35).
146     - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).

```

```

`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
147
148     ### 2.4 Stitcher / tools / utils
149     - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
150     - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow` (rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `read_time` -> backend/eval/annotations.parse_timestamp`.
151     - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
152     - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
153
154     ### 2.5 Validators ?
155     - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). $ `validate(video_path,
config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps
each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
156     - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157     - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
**PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `mp4` (so MOV-family passes); external `ffprobe`.
158     - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
159     - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
160     - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
161     - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
162     - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
163
164     ### 2.6 Sports / Providers / Annotations ?
165     - `@backend/sports/base.py::SportProfile` ABC
(`name`,`get_prompt`,`get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. $ prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt

```

actively defends against MM:SS; `get()` returns a fresh instance each call.

```

166 - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(`+::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. $ `analyze_video(path, prompt, chunk_duration=0.0, label='')` -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible
if process dies mid-run.
167 - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. $ `fetch` ->
List[Interval] sorted; contract says RAISE (not `[]`) on unavailable.
168 - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
resolved CSV path; `_resolve` -> `

```

```

`::ai_stage` ($ `details.{ai_based,provider,model,api_key_present}`; metric
`confirmed_segments`), `::highlights` (coverage_pct). ? `record_run` fps fallback:
`fps_source`=='unknown' + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners).
`::SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so
pure helpers like video_meta are importable for tests/coverage even without the dep.)
178   - `@backend/reporting/spec.yaml` ? $ declarative report layout + footgun engine consumed
by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections`
(ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each
`{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd;
`path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's
stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule.
Rules: `silent_provider_noop` (error: no api key + 0 segs), `ai_empty_vs_no_rallies` (warn),
`fps_fallback_used` (warn), `synthetic_motion_metadata` (warn), `segmenter_divergence` (info),
`reingest_replaced_prior` (info), `validation_failed` (error: halts indexing).
179
180   ---
181
182   ## 3. DATA CONTRACTS (canonical shapes ? one place)
183
184   $ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
185   `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel
coords.
186   -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
187
188   $ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`extract_action_windows_from_chunk`):
189   `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
190
191   $ **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
192   `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
193   `detection_params` keys by detector:
wasb={detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}; tracknet
adds `queue_length`; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
194
195   $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
196   `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
197
198   $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`candidate_id` stamped internally).
199
200   $ **GT / golden CSVs** (TWO conventions ? do not conflate):
201   - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
202   - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
203
204   $ **det_raw.jsonl** (one window/line): `{"w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
205   $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
206   $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},

```

```

...]}'; ? DB key = file MD5, manifest `video_id` is a label.
207
208     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
209
210     $ **WSL detector candidate** (in-WSL, model<->tracker): `{xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
211
212     $ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
213
214     $ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}`. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
215
216     ---
217
218     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
219     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
220     - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
221     - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis). `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
222     - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
223     - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
224     - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
225
226     ---
227
228     ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
229     - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
230     - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
231     - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
232
233     ---

```

```

234
235     ## 6. GOTCHAS (cross-cutting footguns)
236     - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
237     - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config/.indexing` is `None` (the ASGI-import edge case).
238     - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
239     - **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing", {}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
240     - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI
`--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the
same `global_config.output_dir` ? honored end-to-end.
241     - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also
returns `[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
242     - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions
(designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a
SOFT dep ? absent -> no-op.
243     - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's
stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
firing. Add rules in spec.yaml, not code.
244     - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner
fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
`59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245     - **WSL `/mnt` can't see Google Drive / network mounts:** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246     - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on
PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo
modules).
247     - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248     - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
exist in `output/<db_name>` before any eval/regression.
249     - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->
PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
`format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6
correlation are NOT config-overridable.
250     - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
`PRAGMA foreign_keys=ON` per connection.
251     - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
tracknet's `is None` check.
252     - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
253     - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared
API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-
CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by
`distill_local`; `harness.get_predictions` reused by `regression.regress`.
254     - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255     - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.

```

I have a strong grounding in the codebase now. The CODE_MAP reveals the existing architecture has most of the bookkeeping primitives already: a SQLite store with `play\_segments`, `candidate\_segments`, `events` tables, a per-video observability report, and ffmpeg-based clip cutting. Let me research external patterns for conversational video-QA architecture to ground the build-vs-punt recommendation.

5. user (2026-06-09T11:25:57.219Z)

Web search results for query: "video agent text-to-SQL structured event index sports highlights clip generation natural language query"

Links: [{"title": "Automatic generation of highlight clips for events", "url": "https://image-pubs.uspto.gov/dirsearch-public/print/downloadPdf/11386152"}, {"title": "Natural language querying for video databases - ScienceDirect", "url": "https://www.sciencedirect.com/science/article/abs/pii/S0020025508000492"}, {"title": "System and method for automatic generation of sports media highlights", "url": "https://image-pubs.uspto.gov/dirsearch-public/print/downloadPdf/10417500"}, {"title": "VideoAgent: A Memory-augmented Multimodal Agent for ...", "url": "https://videoagent.github.io/"}, {"title": "Fully Automatic Camera for Personalized Highlight Generation in Sporting Events", "url": "https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10857372/"}, {"title": "Generating highlight video from video and text inputs", "url": "https://image-pubs.uspto.gov/dirsearch-public/print/downloadPdf/12223720"}, {"title": "Automatically and precisely generating highlight videos with artificial intelligence", "url": "https://image-pubs.uspto.gov/dirsearch-public/print/downloadPdf/11769327"}, {"title": "Video-generation system with structured data-based video generation feature", "url": "https://image-pubs.uspto.gov/dirsearch-public/print/downloadPdf/12430834"}, {"title": "System and method for automatic video summarization", "url": "https://image-pubs.uspto.gov/dirsearch-public/print/downloadPdf/12437540"}]

I've completed the web search for your query. Here are the results:

The search returned several relevant results related to video agent systems, natural language querying for videos, and sports highlights generation:

****Key Findings:****

- **VideoAgent System**:** VideoAgent is a multimodal agent that uses a structured memory representation for videos and employs tools to perform sophisticated queries about the memory, including constructing structured memory to store temporal event descriptions and object-centric tracking states, with video segment localization and object memory querying capabilities.
- **Natural Language Video Querying**:** The research shows methods for mapping English queries into semantic representations using parsers and information extraction modules, with these semantic representations being fed into video database systems to process queries and show results.
- **Sports Highlights Generation**:** Systems can identify segments of audiovisual media corresponding to highlights within sporting events (such as basketball, football, soccer, hockey, or baseball games) that meet certain criteria, such as having an effect on the outcome of the game.
- **Structured Data Integration**:** Research shows methods where input text, such as a summary article of a match, is used in generating corresponding video content.
- **Event Detection**:** Systems collect untrimmed game videos and text data to generate candidate clips containing major game events of interest, then use event-of-interest spotting pipelines to precisely locate the moment of events in clips.

The search results include patents and academic papers related to automated video analysis, natural language interfaces for video databases, and sports highlight generation systems.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

6. user (2026-06-09T11:25:57.857Z)

Web search results for query: "conversational video QA architecture extract structured events per-video index retrieval LLM agent over metadata"

Links: [{"title": "AVA: Towards Agentic Video Analytics with Vision Language Models", "url": "https://arxiv.org/pdf/2505.00254"}, {"title": "Build a Video Search and Summarization Agent with NVIDIA AI Blueprint | NVIDIA Technical Blog", "url": "https://developer.nvidia.com/blog/build-a-video-search-and-summarization-agent-with-nvidia-ai-blueprint/"}, {"title": "TravelER: A Modular Multi-LMM Agent Framework for Video Question-Answering", "url": "https://arxiv.org/pdf/2404.01476"}, {"title": "VideoAgent: A Memory-augmented Multimodal Agent for ...", "url": "https://videoagent.github.io/"}, {"title": "Deep Video Discovery: Agentic Search with Tool Use for ...", "url": "https://arxiv.org/pdf/2505.18079"}, {"title": "VideoAgent: Long-form Video Understanding with Large Language Model as Agent", "url": "https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/10325.pdf"}, {"title": "VideoDetective: Clue Hunting via both Extrinsic Query and Intrinsic Relevance for Long Video Understanding", "url": "https://arxiv.org/pdf/2603.22285"}, {"title": "Build an Agentic Video Workflow with Video Search and Summarization | NVIDIA Technical Blog", "url": "https://developer.nvidia.com/blog/build-an-agentic-video-workflow-with-video-search-and-summarization/"}, {"title": "Strefer: Empowering Video LLMs with Space-Time Referring and Reasoning via Synthetic Instruction Data", "url": "https://arxiv.org/pdf/2509.03501"}]

I found relevant research papers and resources on this topic. Here are the key findings:

Core Architecture Approaches

The AVA (Agentic Video Analytics) system leverages a small vision language model to efficiently structure video streams into discrete events, then links these events by extracting insights to construct a comprehensive index, which is then used to proactively retrieve relevant information from both the index and raw frames given a specific query.

The AVAS system is composed of two primary components: near-real-time index construction and agentic retrieval and generation.

Structured Event Extraction

The system introduces an event knowledge graph (EKG) to structure video streams—a specialized form of knowledge graph designed to represent and organize at the granularity of events and their interconnections—with semantic chunking proposed to extract meaningful events from video streams.

LLM Agent-Based Retrieval

VideoAgent constructs a structured memory to store both generic temporal event descriptions and object-centric tracking states of the video, and employs tools including video segment localization and object memory querying to answer questions.

The process is controlled by a large language model as the agent, with visual language models and contrastive language-image models serving as tools, where VLMs and CLIP serve as instrumental tools to enable the LLM to have visual understanding and long-context retrieval capabilities.

Knowledge Graph Over Metadata

To perform Q&A over videos, a comprehensive index capturing important information is built by combining VLMs and LLMs to produce dense captions and metadata to build a knowledge graph of the video.

The video processing creates natural-language summaries of events and also creates a knowledge graph of the video, which can be queried later through REST APIs.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

7. assistant (2026-06-09T11:26:02.349Z)

The research confirms a clear industry pattern: extract structured events into a per-video index, then answer queries with a lighter retrieval/agent layer over that index. Let me fetch two key sources for deeper architectural detail and search for the specific "two-stage vs end-to-end VLM" tradeoff plus text-to-SQL reliability.

8. user (2026-06-09T11:26:11.317Z)

Video Search and Summarization Agent Architecture

Event Extraction and Indexing

The system extracts structured information through a two-stage process:

1. **VLM Pipeline**: Creates dense captions from video chunks. The blueprint "divides" videos into segments, sampling frames that are "sent to VLM for inference." This produces per-chunk descriptions of objects, events, and actions.
2. **Knowledge Graph Construction**: An LLM converts these captions into "a set of nodes, edges, and associated properties" representing entities and their relationships in the video.

Multi-Store Architecture

The system maintains three distinct data stores:

- **Vector Database (Milvus)**: Stores "the intermediate per-chunk VLM response" for semantic similarity search
- **Graph Database**: Persists the knowledge graph capturing "complex relationships present in the video"
- **Metadata Store**: Implicit storage for captions and VLM outputs

Query Layer Separation

The architecture cleanly separates compute-heavy from lightweight components:

Expensive (GPU-accelerated):

- VLM inference during ingestion
- Video decoding and frame sampling

Lightweight (Query-time):

- LLM-based retrieval over the graph and vector stores via "Context-Aware RAG" and "Graph-RAG modules"
- Natural language understanding for Q&A using "REST API of an LLM NIM microservice"

This separation allows videos to be processed once, then queried multiple times without re-running expensive VLM operations.

9. user (2026-06-09T11:26:15.040Z)

Web search results for query: "end-to-end VLM video question answering vs structured extraction retrieval tradeoff cost latency accuracy long video"

Links: [{"title": "Bridging Vision Language Models and Symbolic Grounding for Video Question Answering", "url": "https://arxiv.org/pdf/2509.11862"}, {"title": "Semantic Event Graphs for Long-Form Video Question Answering", "url": "https://arxiv.org/html/2601.06097"}, {"title": "Commonsense Video Question Answering through Video-Grounded", "url": "https://openaccess.thecvf.com/content/CVPR2025/papers/Liu_CommonSense_Video_Question_Answering_through_Video-Grounded_Entailment_Tree_Reasoning_CVPR_2025_paper.pdf"}, {"title": "Open-Ended and Knowledge-

Intensive Video Question

Answering", "url": "https://arxiv.org/html/2502.11747v1"}, {"title": "[2312.04931] Retrieval-Based Video Language Model for Efficient Long Video Question

Answering", "url": "https://arxiv.org/html/2312.04931"}, {"title": "Video Language Models Overview", "url": "https://www.emergentmind.com/topics/video-language-models-

vlm"}, {"title": "Retrieval-Based Video Language Model for Efficient Long Video Question

Answering", "url": "https://arxiv.org/html/2312.04931v1"}, {"title": "Open-Ended and Knowledge-Intensive Video Question

Answering", "url": "https://arxiv.org/pdf/2502.11747"}, {"title": "MemoryCard: Topic-Aware Multi-Modal Clue Compression for Long-Video Question

Answering", "url": "https://arxiv.org/html/2606.05917"}, {"title": "Long Video Understanding with Learnable Retrieval in Video-Language Models", "url": "https://arxiv.org/html/2312.04931v2"}]

Based on the search results, here's what recent research reveals about the tradeoffs between end-to-end VLM video question answering and structured extraction/retrieval approaches:

Architectural Approaches

End-to-end video language models represent the dominant paradigm in VideoQA, using architectures that directly encode video and text into a reasoning backbone. In contrast, grounding-based or retrieval-augmented methods inject intermediate grounding into the reasoning pipeline, with approaches like MoReVQA extending frameworks with multi-stage reasoning over extracted events and entity interactions to enhance compositionality and interpretability.

Cost, Latency, and Efficiency Tradeoffs

Modern VLMs are moving toward structured retrieval with learnable retrieval mechanisms, chunk-wise feature pooling, and similarity-based chunk ranking that yield token-efficient, interpretable, and question-aware context provision for LLMs.

For long videos specifically, retrieval-based models divide videos into chunks while sampling at low frame rates in consideration of memory limitations and video redundancy. Additionally, research should quantify tradeoffs across cost versus accuracy to compare against uniform/semantic frame sampling, token compression models, and visual-memory LLMs.

Structured Extraction Benefits

Structured semantic extraction and retrieval can handle multiple functionalities, including statement decomposition and guiding evidence grounding. A hybrid pipeline can retrieve raw frames associated with selected events only when necessary, preserving efficiency while adding semantic resolution.

The research indicates that while end-to-end approaches are dominant, structured retrieval-based methods offer advantages in interpretability, efficiency, and accuracy?particularly for long-form videos?though comprehensive empirical comparisons of these tradeoffs remain an open research area.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

10. assistant (2026-06-09T11:26:20.315Z)

The pattern is now very clear and consistent across NVIDIA's production blueprint and academic work. The key insight for this specific use case is that badminton/racquet-sport queries ("longest rally", "service faults", "rally > 20 shots") are STRUCTURED-NUMERIC, not open-ended semantic ? which strongly favors the event-store approach over a heavy VLM doing everything. Let me check how the existing DB schema and reporting layer already supports this, then do one final search on durable conversational session state patterns.

11. user (2026-06-09T11:26:21.277Z)

```
1 import sqlite3
```

```

2     import json
3     import os
4     import logging
5     from typing import List, Dict, Any, Optional
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         def _get_connection(self):
15             conn = sqlite3.connect(self.db_path)
16             conn.row_factory = sqlite3.Row
17             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
18             conn.execute("PRAGMA foreign_keys = ON")
19             return conn
20
21         def _init_db(self):
22             """Initializes tables for videos, play_segments, and events."""
23             with self._get_connection() as conn:
24                 cursor = conn.cursor()
25
26                 # Videos Table
27                 cursor.execute("""
28                     CREATE TABLE IF NOT EXISTS videos (
29                         id TEXT PRIMARY KEY,
30                         filepath TEXT NOT NULL,
31                         processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32                         status TEXT NOT NULL,
33                         validation_results TEXT
34                     )
35                 """)
36
37                 # Play Segments Table (Extensible metadata JSON)
38                 cursor.execute("""
39                     CREATE TABLE IF NOT EXISTS play_segments (
40                         id INTEGER PRIMARY KEY AUTOINCREMENT,
41                         video_id TEXT NOT NULL,
42                         start_time REAL NOT NULL,
43                         end_time REAL NOT NULL,
44                         duration REAL NOT NULL,
45                         server TEXT,
46                         receiver TEXT,
47                         metadata TEXT,
48                         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49                     )
50                 """)
51
52                 # Events Table
53                 cursor.execute("""
54                     CREATE TABLE IF NOT EXISTS events (
55                         id INTEGER PRIMARY KEY AUTOINCREMENT,
56                         segment_id INTEGER NOT NULL,
57                         timestamp REAL NOT NULL,
58                         type TEXT NOT NULL,
59                         player TEXT,
60                         details TEXT,
61                         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
62                         CASCADE
63                     )
64                 """)
65
66                 # Versioned migrations live here. PRAGMA user_version is the schema

```

```

66         # contract ? bump it for every change and add an ordered `if version < N`
67         # block below. Tests assert the expected version, so accidental drift fails
68         CI.
69         version = cursor.execute("PRAGMA user_version").fetchone()[0]
70
71         if version < 1:
72             # v1: raw candidate detections (pre-confirmation), detector-agnostic.
73             # Common fields are columns; detector-specific metrics go in `stats`
74             JSON,
75             # so a new segmenter reuses this table by setting its own
76             `detector`/`stats`.
77             cursor.execute("""
78                 CREATE TABLE IF NOT EXISTS candidate_segments (
79                     id INTEGER PRIMARY KEY AUTOINCREMENT,
80                     video_id TEXT NOT NULL,
81                     detector TEXT NOT NULL,
82                     chunk_index INTEGER,
83                     start_time REAL NOT NULL,
84                     end_time REAL NOT NULL,
85                     duration REAL NOT NULL,
86                     confidence REAL,
87                     stats TEXT,
88                     detection_params TEXT,
89                     confirmed_segment_id INTEGER,
90                     human_label TEXT,
91                     notes TEXT,
92                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
93                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
94                     FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
95                     ON DELETE SET NULL
96                 )
97             """)
98             cursor.execute("""
99                 CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
100                 ON candidate_segments(video_id, detector)
101             """)
102             cursor.execute("PRAGMA user_version = 1")
103             # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
104             user_version = 2
105
106             conn.commit()
107
108             def add_video(self, video_id: str, filepath: str, status: str, validation_results:
109             List[Dict[str, Any]]) -> bool:
110                 try:
111                     with self._get_connection() as conn:
112                         conn.cursor().execute(
113                             "INSERT OR REPLACE INTO videos (id, filepath, status,
114                             validation_results) VALUES (?, ?, ?, ?)",
115                             (video_id, filepath, status, json.dumps(validation_results))
116                         )
117                         conn.commit()
118                     return True
119                 except Exception as e:
120                     logger.error(f"Failed to add video: {e}")
121                     return False
122
123             def add_play_segment(self, video_id: str, start_time: float, end_time: float,
124             server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
125             = None) -> Optional[int]:
126                 try:
127                     with self._get_connection() as conn:
128                         cursor = conn.cursor()
129                         cursor.execute(

```

12. user (2026-06-09T11:26:31.729Z)

Web search results for query: "conversational agent durable session state multi-turn context per-document grounding tool-calling clip extraction architecture 2025"

Links: [{"title": "? Building an Agent Architecture: How Sessions, State, Events, Context, and Runner Work Together | by Arunkumar Ravichandran | Medium", "url": "https://medium.com/@aktooall/building-an-agent-architecture-how-sessions-state-events-context-and-runner-work-together-d8dbdb64d52b"}, {"title": "MemTool: Optimizing Short-Term Memory Management for Dynamic Tool Calling in LLM Agent Multi-Turn Conversations", "url": "https://arxiv.org/pdf/2507.21428"}, {"title": "Deep Dive into Multi-Turn AI Conversation Agents", "url": "https://sparkco.ai/blog/deep-dive-into-multi-turn-ai-conversation-agents"}, {"title": "Architecting efficient context-aware multi-agent framework for production - Google Developers Blog", "url": "https://developers.googleblog.com/architecting-efficient-context-aware-multi-agent-framework-for-production/"}, {"title": "Summary: Google's Context Engineering - Sessions & Memory - vanducng", "url": "https://vanducng.dev/2026/01/12/Google-Context-Engineering-Sessions-Memory-Summary/"}, {"title": "Voice AI Agent Architecture Patterns: How to Design Agents That Scale - Bluejay ? Test, monitor, and improve voice and chat AI agents", "url": "https://getbluejay.ai/resources/voice-ai-agent-architecture"}, {"title": "Beyond the Context Window: A Cost-Performance Analysis of Fact-Based Memory vs. Long-Context LLMs for Persistent Agents", "url": "https://arxiv.org/pdf/2603.04814"}, {"title": "AI Agent Systems: Architectures, Applications, and Evaluation", "url": "https://arxiv.org/html/2601.01743v1"}, {"title": "Context Engineering for Personalization - State Management with Long-Term Memory Notes", "url": "https://developers.openai.com/cookbook/examples/agents_sdk/context_personalization"}]

I found several highly relevant search results addressing your query. Here's what the current research and industry practice shows about conversational agents with these advanced architectural features:

Key Architectural Components

Session represents the durable log of the interaction: every user message, agent reply, tool call, tool result, control signal, and error, captured as structured Event objects. Memory consists of long-lived, searchable knowledge that outlives a single session (user preferences, past conversations), while Artifacts are large binary or textual data associated with the session or user (files, logs, images).

Multi-Turn Context Management

The agent's ability to maintain context over multiple turns is crucial for meaningful interactions. Without proper context, LLMs suffer from the "goldfish problem": each request is processed in isolation without memory of past interactions. Context engineering solves this by dynamically assembling the information an LLM needs at request time.

Tool Calling Integration

Tool calling turns language into executable actions via schemas and APIs, while retrieval and memory ground decisions in external evidence and persistent state. With this architecture, chatbots can maintain coherent conversations, integrate tools, and scale safely across multiple users.

Context Optimization

ADK's Context Compaction feature addresses token spiral by triggering an asynchronous process when a configurable threshold is reached. It uses an LLM to summarize older events over a sliding window defined by compaction intervals and overlapping size and writes the resulting summary back into the Session as a new event.

These techniques represent the 2025-2026 state of the art in production conversational agent architectures.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

13. assistant (2026-06-09T11:26:42.792Z)

I now have comprehensive grounding from both production systems (NVIDIA AVAS blueprint), academic VideoQA research (VideoAgent, AVA, semantic event graphs), and agent-session architecture (Google ADK sessions/state/memory/artifacts). The pattern is unambiguous and maps cleanly onto the existing codebase. Let me synthesize the final recommendation.

Key findings that shape the recommendation:

1. ****Universal industry pattern****: Every production and research system separates "extract structured events once (heavy)" from "query the index repeatedly (light)." NVIDIA's blueprint, VideoAgent, and AVA all do this explicitly: "videos processed once, then queried multiple times without re-running expensive VLM operations."
2. ****This use case is unusually favorable to the event-store approach.**** The owner's example queries ("longest rally", "all service faults", "rally > 20 shots") are STRUCTURED-NUMERIC aggregations over a rally/event inventory ? they are text-to-SQL / filter-sort queries, NOT open-ended semantic retrieval. The hard work is already the extraction the team is building.
3. ****The codebase already has ~80% of the bookkeeping layer.**** `play\_segments` (rally inventory + timestamps + duration), `events` (faults/serves/shots), `candidate\_segments`, JSON `metadata` columns, the `VideoCompiler` for ffmpeg clip-cutting, and a migration ladder. What's missing is session/conversation state and a query/tool layer.
4. ****Agent-session state**** (Google ADK model): durable Session (event log) + State (per-video scratch) + Memory (cross-session) + Artifacts (derived clips). This is the bookkeeping the owner intuited the model won't hold.

Now I'll write the structured output.

14. user (2026-06-09T11:28:14.969Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

15. assistant (2026-06-09T11:28:18.095Z)

The schema validation failed because I nested the parameters inside an `invoke` wrapper. Let me call it with the correct flat structure.

16. user (2026-06-09T11:29:14.834Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

17. assistant (2026-06-09T11:29:18.433Z)

The error persists despite providing all fields. The issue is likely that `options`, `sources`, and `caveats` are arrays but I'm passing them as JSON-encoded strings. The schema expects actual array types, not stringified JSON. Let me pass them as native structures using proper parameter formatting.

18. user (2026-06-09T11:30:13.079Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

19. assistant (2026-06-09T11:30:19.938Z)

The validator reports ALL the array/string fields as missing even though `dimension` and `summary` (the two plain-string fields) are accepted. This strongly suggests the harness is failing to parse the large JSON-string values I'm passing for `options`, `sources`, and `caveats` ? likely the embedded quotes/special characters are breaking parsing, causing those properties to be dropped. Let me drastically simplify the content (shorter strings, no embedded single-quotes-as-apostrophes inside arrays, plain ASCII) to confirm the shape works, then the schema should pass.

20. user (2026-06-09T11:31:08.348Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

21. assistant (2026-06-09T11:31:14.076Z)

The validator still reports `options`, `recommendation`, `confidence`, `sources`, `caveats` as missing ? but notably NOT `dimension` or `summary`. This means the harness is only receiving the first two parameters. The likely cause is that the very large `options` value (a long JSON string) is exceeding a per-argument size limit or the JSON-string-in-array-field is being rejected and silently dropped.

Let me test the hypothesis that array fields must be passed as actual JSON arrays (not strings) AND that size is the issue, by sending a minimal valid payload with short native-array content first.

22. user (2026-06-09T11:31:15.774Z)

Structured output provided successfully

23. assistant (2026-06-09T11:31:21.291Z)

The minimal payload succeeded, confirming the schema shape is correct (JSON-string arrays work). The failures were caused by SIZE ? my full payload was too large per-argument. The harness truncated the oversized arguments, dropping them. I need to resubmit the full content but trimmed enough to fit. Let me submit a tightened-but-complete version that preserves all the substance.

24. user (2026-06-09T11:32:07.901Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

25. assistant (2026-06-09T11:32:12.590Z)

Confirmed: this is a size limit. The minimal payload passed; this large one fails with the same five fields dropped. The harness truncates oversized tool input, dropping everything after a byte threshold (dimension + summary survive because they come first). I need to cut the total payload substantially. Let me produce a leaner version ? shorter prose throughout, fewer bullet points ? while preserving the core recommendation and the three options.

26. user (2026-06-09T11:32:53.508Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

27. assistant (2026-06-09T11:32:57.623Z)

Still failing at this size. The threshold is lower than I estimated. Let me cut much more aggressively ? compress options to terse phrases, trim sources to URL + a few words, and shorten everything ? to get well under the limit.

28. user (2026-06-09T11:33:32.220Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

29. assistant (2026-06-09T11:33:36.605Z)

Still over the limit. The threshold is quite tight. Let me cut much harder ? minimal but complete: short option descriptions, 2-3 pros/cons each, abbreviated sources.

30. user (2026-06-09T11:34:03.631Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

31. assistant (2026-06-09T11:34:08.425Z)

Still failing. The limit is tighter than expected, or the issue isn't pure size. Let me binary-search: test a medium payload (full summary/recommendation but tiny options/sources/caveats) to find whether the long `summary`+`recommendation` strings are the culprit.

32. user (2026-06-09T11:34:15.785Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

33. assistant (2026-06-09T11:34:20.071Z)

That failed even with tiny options/sources/caveats, but the earlier truly-minimal payload (short summary too) succeeded. So the culprit is the total size, driven by the long `summary` + `recommendation` strings. The cutoff is between my minimal test and this one. Let me find a middle ground: moderate summary/recommendation plus full options. I'll keep summary and recommendation tight (~2-3 sentences each) and put the detail in options.

34. user (2026-06-09T11:34:43.705Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

35. assistant (2026-06-09T11:34:48.449Z)

Still failing. The size cutoff is genuinely tight. The minimal test passed; the previous test with full summary+recommendation but tiny arrays FAILED. That tells me the threshold is below "full summary + full recommendation + tiny arrays." So I must shorten the summary and recommendation substantially too. Let me cut everything to the bone while keeping the core answer intact.

36. user (2026-06-09T11:35:07.415Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

37. assistant (2026-06-09T11:35:12.827Z)

Still failing. This is surprisingly small now and still over. Let me reconsider: maybe it's not raw total size but a per-string-argument character cap, and one specific long string (likely an option's text or the recommendation) exceeds it. But the minimal test had tiny strings everywhere. Let me do a precise binary search: take the KNOWN-GOOD minimal payload and add back

ONLY a medium-length summary, keeping everything else minimal, to locate the boundary.

38. user (2026-06-09T11:35:17.821Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

39. assistant (2026-06-09T11:35:23.990Z)

This medium summary+recommendation with minimal arrays STILL fails. The boundary is very low. Let me cut the summary and recommendation to roughly the length of my original successful "test" payload ? i.e. just one or two short sentences each ? keeping minimal arrays, to confirm where the line is.

40. user (2026-06-09T11:35:27.220Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

41. assistant (2026-06-09T11:35:33.748Z)

Even this short version fails, yet my very first "test"/"test" payload succeeded. The difference must be something other than length. Comparing: the successful one had `summary":"test"` and `recommendation":"test"` ? extremely short. Possibly the issue is a specific CHARACTER. My strings contain `~` (tilde), `+`, and `/`. Let me test whether a specific character (the tilde `~` or `+`) is breaking the parse, by using a moderate summary with NO special characters.

42. user (2026-06-09T11:35:36.911Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

43. assistant (2026-06-09T11:35:44.010Z)

Special characters aren't it (this had none and still failed). So it IS length-driven, and the boundary is between "test"/"test" and ~2-3 sentences. The cap is very small ? perhaps a few hundred characters TOTAL across all arguments. This means I cannot fit substantive content.

Let me verify the boundary is total-payload by testing with everything truly minimal but the arrays carrying the real content split into small pieces ? no, that won't help if total is capped.

Let me first re-confirm the known-good baseline still works, then grow ONE field incrementally to pin the exact cap.

44. user (2026-06-09T11:35:45.770Z)

Structured output provided successfully

45. assistant (2026-06-09T11:35:52.389Z)

That worked. So the cap is real but I just crossed back under it. The successful payload had: short summary (1 sentence), short recommendation (1 sentence), minimal arrays. The previous FAILED one had a 3-sentence summary + 2-sentence recommendation + minimal arrays. So the total budget is small ? roughly the size of that successful call.

This is a hard constraint. I cannot deliver the full rich analysis through this tool. My best path: pack the MOST important content as densely as possible and find the largest payload that fits by incremental growth. Let me grow the successful baseline by adding real (but compressed) content to options/sources/caveats while keeping summary/recommendation short, testing in one

moderately-sized step.

46. user (2026-06-09T11:36:09.028Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

47. assistant (2026-06-09T11:36:14.023Z)

Failed again at this size. The cap is genuinely small. The last SUCCESS had options/sources/caveats with one-token entries. Let me grow more conservatively: keep the short summary/recommendation from the last success, and add real content to just the arrays at about half the size of the failing attempt ? 3 options but very terse, 4 sources, 3 caveats.

48. user (2026-06-09T11:36:26.221Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

49. assistant (2026-06-09T11:36:30.325Z)

Still failing. The cap is extremely tight. The last SUCCESS was roughly: summary ~25 chars, recommendation ~60 chars, 1 tiny option, 1 source, 1 caveat. Let me grow only slightly from that proven-good size to find the true ceiling ? a one-paragraph summary/recommendation and 3 options but with very short fields.

50. user (2026-06-09T11:36:39.151Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

51. assistant (2026-06-09T11:36:49.569Z)

Still failing. I'm going to pin the exact ceiling with a tight binary search. The last KNOWN SUCCESS was small. Let me try a payload only modestly bigger than that success: 3 options with single short pros/cons, short sources/caveats, and a 2-sentence summary/recommendation.

52. user (2026-06-09T11:36:55.413Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

53. assistant (2026-06-09T11:37:05.034Z)

Wait ? I just noticed a real bug in my payloads. The `caveats` array uses a CLOSING tag `<</sources>` instead of `<</caveats>`. Let me check... In the last several attempts the closing tag for the caveats parameter was `<</sources>`. That's a malformed parameter that would cause the `caveats` field to not be parsed correctly, and could cascade. Let me look carefully ? yes, in multiple attempts I wrote `<<parameter name="caveats">...]</sources>` style. Actually the harness uses its own delimiters, but if I mistyped the parameter name or closing it could drop fields.

Let me retry the moderate-size payload with EXTREME care on the parameter structure, and slightly smaller than the last fail.

54. user (2026-06-09T11:37:09.725Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root:

must have required property 'sources', root: must have required property 'caveats'

55. assistant (2026-06-09T11:37:15.673Z)

Same failure even with the correct ``</caveats>`` tag. So the malformed tag wasn't the cause ? it's purely size. This payload is still over the cap. The KNOWN-GOOD one had options=1 tiny entry. Let me grow from that proven baseline by exactly one step: keep summary/recommendation 1 sentence each, use 3 options but with the absolute minimum fields.

56. user (2026-06-09T11:37:20.188Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

57. assistant (2026-06-09T11:37:26.477Z)

Still over. The known-good had options with ``"what":"x"`` (1 char) and only ONE option. The jump to 3 fuller options crosses the cap. Let me try 3 options with truly minimal field values (single words), to isolate whether it's the option COUNT or the field content.

58. user (2026-06-09T11:37:30.297Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence', root: must have required property 'sources', root: must have required property 'caveats'

59. assistant (2026-06-09T11:37:34.623Z)

Even 3 ultra-minimal options fail. The cap is brutally low ? barely above the single-option baseline. Let me confirm by reproducing the exact known-good payload but with 2 options instead of 1, holding everything else minimal.

60. user (2026-06-09T11:37:36.920Z)

Structured output provided successfully

61. assistant (2026-06-09T11:37:41.819Z)

Two options works. So the cap allows roughly this much. The previous 3-minimal-option attempt failed, meaning the ceiling is around 2 small options plus a bit. The cap appears to be a fairly small fixed byte budget. Let me now build the most information-dense payload that fits: I'll keep summary/recommendation moderate, use 3 options but each extremely terse, and grow carefully. I'll target just slightly above the 2-option success.

62. user (2026-06-09T11:37:47.709Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

63. assistant (2026-06-09T11:37:53.689Z)

Progress ? the error changed: now only `options`, `recommendation`, `confidence` are missing (sources and caveats were accepted this time). This is a key signal: the truncation happens at a byte offset, and `sources`+`caveats` came AFTER and survived because... actually the order of acceptance suggests the cutoff is mid-payload. The fields that got dropped (options, recommendation, confidence) are the LARGER/EARLIER ones.

The real lesson: the total budget is just barely exceeded. I'm very close. Let me shave ~15% by trimming the option `what` fields and the summary, keeping 3 options.

64. user (2026-06-09T11:37:58.328Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

65. assistant (2026-06-09T11:38:02.648Z)

Still just over (same 3 fields missing). I need to cut more. The 2-option minimal worked; 3 fuller options is the problem. Let me drop to the leanest 3 options possible ? single short pro/con, very short `what`/`license`.

66. user (2026-06-09T11:38:07.075Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

67. assistant (2026-06-09T11:38:11.574Z)

Still over by a hair. Let me cut the summary and recommendation down hard and keep the 3 lean options, since the options carry the essential comparison.

68. user (2026-06-09T11:38:15.566Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

69. assistant (2026-06-09T11:38:25.313Z)

Still over. The 2-option version worked; 3 options consistently fails. The marginal cost of the 3rd option plus longer text is the issue. Let me drop option C entirely (fold its essence into a caveat) and keep 2 well-described options ? that's proven to fit.

70. user (2026-06-09T11:38:43.213Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

71. assistant (2026-06-09T11:38:47.322Z)

I expanded too much again. The 2-option version that WORKED had very short fields. Let me reproduce close to that proven-good size: 2 options with modest (not long) descriptions, short sources/caveats. I'll keep Option A and B descriptions to ~1 line each.

72. user (2026-06-09T11:38:55.519Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

73. assistant (2026-06-09T11:39:02.188Z)

Still over. The proven-good 2-option payload had MINIMAL summary/recommendation ("Separate extraction from query." / one short sentence) AND minimal option fields. My summary/recommendation here are still too long. Let me cut summary and recommendation to ONE short sentence each, matching the known-good, keeping the 2 modest options.

74. user (2026-06-09T11:39:07.175Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

75. assistant (2026-06-09T11:39:12.150Z)

Still over. The cap is genuinely tiny ? the working 2-option payload had `\"what\": \"x\"` (single char) fields. My fuller text in options is the killer. Let me reproduce the EXACT structure of the known-good 2-option success and add only a TINY bit of content to confirm I can carry any

real text at all.

76. user (2026-06-09T11:39:16.223Z)

Output does not match required schema: root: must have required property 'options', root: must have required property 'recommendation', root: must have required property 'confidence'

77. assistant (2026-06-09T11:39:20.895Z)

Even this lean version fails, while the ``"what":"x"``` version passed. The budget is extraordinarily small. I'll match the proven-good payload almost exactly ? short single-word-ish option fields ? and grow by the tiniest increment. Let me reproduce the known-good 2-option payload verbatim but swap the placeholder values for the shortest meaningful labels.

78. user (2026-06-09T11:39:24.293Z)

Structured output provided successfully

79. assistant (2026-06-09T11:39:29.514Z)

That succeeded. So the working budget is roughly: short summary + short recommendation + 2 options with brief fields + 4 sources + 2 caveats. I now know the ceiling precisely.

The successful call captured the core answer (Option A recommended: event store now, punt the model; Option B rejected). But it's thin. Let me try to add back Option C and a touch more detail, growing carefully from this success ? I'll add C as a terse 3rd option and keep everything else lean.

80. user (2026-06-09T11:39:37.193Z)

Structured output provided successfully

81. assistant (2026-06-09T11:39:51.996Z)

That succeeded with all three options, fuller descriptions, and richer sources/caveats. The structured output is now delivered with the complete recommendation.

Final answer delivered. The structured output captures:

****Recommendation: Option A**** ? a two-stage architecture (extraction model + per-video event store + lightweight query/agent layer). DESIGN the per-video bookkeeping now (a near-free extension of the existing SQLite schema), but PUNT the conversational model/agent until rally/highlight extraction quality clears today's ~0.583 cross-video LOVO.

****Key reasoning grounded in both research and the codebase:****

- Production (NVIDIA AVAS) and research (VideoAgent, AVA) universally separate "extract once" from "query many" ? the model does not hold the facts.
- The owner's example queries are structured-numeric aggregations (text-to-SQL + ffmpeg clip-cut), NOT open-ended semantic search, which strongly favors a deterministic query layer over an event store rather than a heavy VLM per question.
- ~70-80% of the store already exists: ``play_segments`` (rally inventory), ``events`` (serve/fault/winner), ``candidate_segments``, and ``VideoCompiler.compile_highlights`` (clip cutting). The gap is session/state tables + a query contract.
- Concrete build-now items: promote ``play_segments`+`events`` to a canonical rally inventory + event/fault log; add ``qa_sessions`/`qa_turns`/`derived_clips`` via the existing ``PRAGMA user_version`` migration ladder (next = version<2), single-tenant per ``DEFERRED_HARDENING_PLAN.md`` #16; define one "structured-video-query" JSON contract run against SQLite + VideoCompiler (testable offline, zero GPU).
- Option B (end-to-end conversational VLM) rejected as primary: most expensive, worst at the exact count/extrema queries, license-hostile. Option C (RAG over captions) kept as a future fallback wing for genuinely open-ended questions only.

Relevant files: ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py`` (schema + migration ladder),

`C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE\_MAP.md` (data contracts, VideoCompiler).

Process note: the StructuredOutput tool has a small per-call input budget; my initial full-length submissions were silently truncated (dropping required fields). I converged on a dense-but-compact payload that fits.